

智慧的策略

贪心、并查集与数学 · 9题 · C++ & Python 双语对照

NQ163 区间选点 AcWing 905

给定N个闭区间 $[a_i, b_i]$ ，请在数轴上选择尽量少的点，使得每个区间内至少包含一个选出的点。输出选择的点的最小数量。位于区间端点上的点也算作区间内。

输入

第一行包含整数N，表示区间数。接下来N行，每行包含两个整数 a_i, b_i ，表示一个区间的两个端点。数据范围 $1 \leq N \leq 10^5$, $-10^9 \leq a_i \leq b_i \leq 10^9$

输出

输出一个整数，表示所需的点的最小数量。

来源

AcWing 905

输入

3
-1 1
2 4
3 5

输出

2

提示 [原题链接](#) [Y总代码](#) [参考题解](#) [Y总讲解](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

struct Range
{
    int l, r;
    bool operator< (const Range & R) const
    {
        return r < R.r; //重载比较符, 根据r排序
    }
} range[N];

int main()
{
    int n;
    scanf("%d", &n);
    //读入区间
    for (int i = 0; i < n; i++)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        range[i] = {l, r};
    }
    //按照右端点排序
    sort(range, range+n);
    //根据右端点计算覆盖的区间个数
    int res = 0, ed = -2e9; //已统计区间的右端点
    for (int i = 0; i < n; i++)
    {
        if (range[i].l > ed)
        {
            res++;
            ed = range[i].r;
        }
    }
    printf("%d\n", res);

    return 0;
}

```

Python

Python solution pending

NQ164 合并果子 AcWing 148

在一个高科技果园中，林克利用先进的无人机将所有果子采集完毕，并通过智能分类系统按照果子的种类将它们分成了不同的堆。林克决定通过纳米融合技术将所有果子合成一堆。每次融合，林克可以选择两堆果子进行合并，消耗的能量等于两堆果子的重量之和，能量单位由果园的中央AI计算并记录。显然，经过 $n-1$ 次融合后，所有果子将整合为一堆。林克在融合果子时总共消耗的能量等于每次融合所耗能量之和。由于林克还需要驾驶悬浮运输机将这些果子运回基地，他希望在融合过程中尽可能节省能量，以便优化运输效率。假设每个果子的重量为1个标准单位，并且已知果子的种类数和每种果子的数量，你的任务是设计出一种融合次序方案，使林克消耗的能量最少，并输出这个最小的能量值。例如，有3种果子，数量依次为1、2、9。林克可以先将1和2堆融合，新堆数量为3，消耗能量为3。接着，将新堆与原先的第3堆融合，得到数量为12的新堆，消耗能量为12。因

此，林克总共消耗的能量为 $3+12=15$ 。可以证明，15是可能的最小能量值。数据范围： $1 \leq n \leq 10000, 1 \leq a_i \leq 20000$ 请根据此问题，计算出林克最小的能量消耗值，并说明你的方案。

输入 输入包括两行，第一行是一个整数 n ，表示果子的种类数。第二行包含 n 个整数，用空格分隔，第 i 个数 a_i 是第 i 种果子的数目。

输出 输出包括一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31} 。

来源 AcWing 148

输入	输出
3	15
1 2 9	

提示 原题链接 [acwing讲解](#)

C++

```
#include <iostream>
#include <queue>

using namespace std;

int main()
{
    int n; cin>>n;
    //优先队列，使用greater作比较函数，把默认的大根堆变为小根堆
    priority_queue<int, vector<int>, greater<int>> heap;
    //把读入数据压入堆
    while (n--)
    {
        int x; cin>>x;
        heap.push(x);
    }

    int res = 0;
    while(heap.size() > 1)
    {
        int x = heap.top(); heap.pop();
        int y = heap.top(); heap.pop();
        res += x + y; //累计合并值
        heap.push(x+y); //把当前合并的值压入堆
    }

    cout<<res<<endl;

    return 0;
}
```

Python

Python solution pending

NQ165 并查集试炼之合并集合 AcWing 836

一共有 n 个数，编号是 $1 \sim n$ ，最开始每个数各自在一个集合中。现在要进行 m 个操作，操作共有两种：“M a b”，将编号为 a 和 b 的两个数所在的集合合并，如果两个数已经在同一个集合中，则忽略这个操作；“Q a b”，询问编号为 a 和 b 的两个数是否在同一个集合中；数据范围： $1 \leq n, m \leq 10^5$

输入 第一行输入整数n和m。接下来m行，每行包含一个操作指令，指令为“M a b”或“Q a b”中的一种。

输出 对于每个询问指令“Q a b”，都要输出一个结果，如果a和b在同一集合内，则输出“Yes”，否则输出“No”。每个结果占一行。

来源 AcWing 836

输入	输出
4 5	Yes
M 1 2	No
M 3 4	Yes
Q 1 2	
Q 1 3	
Q 3 4	

提示 [原题链接](#) [Y总代码](#) [Y总讲解](#)

C++

```
#include <iostream>
using namespace std;
const int N = 100007;
int p[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

void merge(int a, int b)
{
    p[find(a)] = find(b);
}

void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}

int main()
{
    int m;
    cin >> n >> m; //输入n
    //创造并查集数
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m--)
    {
        char op;
        int a, b;
        cin >> op;
        cin >> a >> b;
        if (op == 'M') merge(a, b);
        else if (op == 'Q') query(a, b);
    }
    return 0;
}
```

Python

Python solution pending

NQ166 并查集试炼之连通块中点的数量 AcWing 837

给定一个包含 n 个点（编号为 $1\sim n$ ）的无向图，初始时图中没有边。现在要进行 m 个操作，操作共有三种：“C a b”，在点 a 和点 b 之间连一条边， a 和 b 可能相等；“Q1 a b”，询问点 a 和点 b 是否在同一个连通块中， a 和 b 可能相等；“Q2 a”，询问点 a 所在连通块中点的数量；数据范围： $1\leq n, m\leq 10^5$

输入 第一行输入整数 n 和 m 。接下来 m 行，每行包含一个操作指令，指令为“C a b”，“Q1 a b”或“Q2 a”中的一种。

输出 对于每个询问指令“Q1 a b”，如果 a 和 b 在同一个连通块中，则输出“Yes”，否则输出“No”。对于每个询问指令“Q2 a”，输出一个整数表示点 a 所在连通块中点的数量 每个结果占一行。

来源 AcWing 837

输入	输出
5 5	Yes
C 1 2	2
Q1 1 2	3
Q2 1	
C 2 5	
Q2 5	

提示 原题 Y总代码 Y总讲解

C++

```

#include <iostream>
#include <cstring>

using namespace std;
const int N = 100007;
int p[N], Size[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}
void merge(int a, int b)
{
    Size[find(b)] += Size[find(a)];
    p[find(a)] = find(b);
}
void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}
int count(int x)
{
    return Size[find(x)];
}
int main()
{
    int m;
    cin >> n >> m; //输入n
    //创建并查集数
    for (int i = 1; i <= n; i++) p[i] = i, Size[i] = 1;
    while (m--)
    {
        string op;
        int a, b;
        cin >> op;
        if (op == "C")
        {
            cin >> a >> b;
            if (find(a) == find(b)) continue; //已经在同一个集合里,
            //跳过merge操作
            merge(a, b);
        }
        else if (op == "Q1") cin >> a >> b, query(a, b);
        else cin >> a, cout << count(a) << endl;;
    }
    return 0;
}

```

Python

Python solution pending

NQ167 并查集试炼之食物链 AcWing 240

动物王国中有三类动物A,B,C，这三类动物的食物链构成了有趣的环形。A吃B，B吃C，C吃A。现有N个动物，以1—N编号。每个动物都是A,B,C中的一种，但是我们并不知道它到底是哪一种。有人用两种说法对这N个动物所构成的食物链关系进行描述：第一种说法是“1 X Y”，表示X和Y是同类。第二种说法是“2 X Y”，表示X吃Y。此人对N个动物，用上述两种说法，一句接一句地说出K句话，这K句话有的是真的，有的是假的。当一句

话满足下列三条之一时，这句话就是假话，否则就是真话。 1) 当前的话与前面的某些真的话冲突，就是假话； 2) 当前的话中X或Y比N大，就是假话； 3) 当前的话表示X吃X，就是假话。 你的任务是根据给定的N和K句话，输出假话的总数。 数据范围 $1 \leq N \leq 50000$, $0 \leq K \leq 100000$

输入

第一行是两个整数N和K，以一个空格分隔。 以下K行每行是三个正整数 D, X, Y，两数之间用一个空格隔开，其中D表示说法的种类。 若D=1，则表示X和Y是同类。 若D=2，则表示X吃Y。

输出

只有一个整数，表示假话的数目。

来源

AcWing 240

输入

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出

```
3
```

提示 原题链接

C++

```

#include <iostream>

using namespace std;

const int N = 50007; // Andy风格

int n, m;
int p[N], d[N]; // p集合 d距离初始化为0

int find(int x)
{
    if (p[x] != x)
    {
        int t = find(p[x]);
        d[x] += d[p[x]];
        p[x] = t;
    }
    return p[x];
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++) p[i] = i; // 初始化p集合

    int res = 0;
    while (m --)
    {
        int t, x, y;
        scanf("%d%d%d", &t, &x, &y);

        if (x > n || y > n) res ++;
        else
        {
            int px = find(x), py = find(y); // 两个集合的父节点
            if (t == 1)
            {
                if (px == py && (d[x] - d[y]) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] - d[x];
                }
            }
            else
            {
                if (px == py && (d[x] - d[y] - 1) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] + 1 - d[x];
                }
            }
        }
    }

    printf("%d\n", res);
    return 0;
}

```

Python

Python solution pending

NQ168 快速幂 AcWing 875

给定 n 组 a_i, b_i, p_i ，对于每组数据，求出 $a^b \bmod p$ 的值。

输入 第一行包含整数 n 。接下来 n 行，每行包含三个整数 a_i, b_i, p_i 。数据范围 $1 \leq n \leq 100000$ $1 \leq a_i, b_i, p_i \leq 2 \times 10^9$

输出 对于每组数据，输出一个结果，表示 $a^b \bmod p$ 的值。每个结果占一行。

来源 AcWing 875

输入	输出
5	2613
1326 8312 7553	3520
1550 7708 3792	1785
500 6795 6235	7244
6880 7895 7532	5346
1904 9984 7790	

提示 [原题链接](#) [Y总讲解](#)

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
typedef long long LL;
int qmi(int a, int b, int p)
{
    int res = 1; //初始值为1
    while (b)
    {
        if (b & 1) //b的最末尾为1
            res = (LL)res * a % p; //必须用long long防止溢出
        b >>= 1; //去掉b最低位
        a = (LL)a * a % p;
    }
    return res;
}

int main()
{
    int n;
    scanf("%d", &n); //数据很大，需要用LL读入
    while (n--)
    {
        int a, b, p;
        scanf("%d%d%d", &a, &b, &p); //读入a b p

        printf("%d\n", qmi(a, b, p)); //计算快速幂
    }
    return 0;
}
```

Python

Python solution pending

NQ169 筛质数 AcWing 868

给定一个正整数 n ，请你求出 $1\sim n$ 中质数的个数。

输入 共一行，包含整数 n 。数据范围 $1\leq n\leq 10^6$

输出 共一行，包含一个整数，表示 $1\sim n$ 中质数的个数。

来源 AcWing 868

输入

8

输出

4

提示 [原题链接](#)

C++

```
// Andy 2021.06.04
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100000007;

int primes[N], cnt;
bool st[N];
void get_primes(int n) // 线性筛质数
{
    for (int i = 2; i <= n; i++)
    {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

int main()
{
    int n; cin >> n;
    get_primes(n);
    cout << cnt << endl;
    return 0;
}
```

Python

```
# Python solution pending
```

NQ170 货仓选址 AcWing 104

在一条数轴上有 N 家商店，它们的坐标分别为 $A_1\sim A_N$ 。现在需要在数轴上建立一家货仓，每天清晨，从货仓到每家商店都要运送一车商品。为了提高效率，求把货仓建在何处，可以使得货仓到每家商店的距离之和最小。数据范围 $1\leq N\leq 100000$

输入 第一行输入整数 N 。第二行 N 个整数 $A_1\sim A_N$ 。

输出 输出一个整数，表示距离之和的最小值。

来源 AcWing 104

输入

4
6 2 9 1

输出

12

提示 原题链接 [acwing讲解](#)

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n;
int a[N];

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(a, a+n);
    int res = 0;
    //中位数a[n/2],求距离之和的最小值
    for (int i = 0; i < n; i++) res += abs(a[i] - a[n/2]);
    cout << res << endl;
    return 0;
}
```

Python

Python solution pending

NQ171 AcWing NQ099 AcWing NQ099

AcWing NQ099

输入 见原题

输出 见原题

来源 AcWing NQ099

C++

// AcWing NQ099

Python

```

# NQ099 最省赛程 - 改进版
# Python 3.5 兼容
import heapq

MAX_N = 1010
MAX_CAPACITY = 107

def main():
    import sys
    input = sys.stdin.read().split()
    idx = 0

    n = int(input[idx])
    idx += 1
    m = int(input[idx])
    idx += 1

    oil_price = [0] * n
    for i in range(n):
        oil_price[i] = int(input[idx])
        idx += 1

    # 构建邻接表
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = int(input[idx])
        idx += 1
        v = int(input[idx])
        idx += 1
        d = int(input[idx])
        idx += 1
        adj[u].append((v, d))
        adj[v].append((u, d))

    q = int(input[idx])
    idx += 1

    for _ in range(q):
        c = int(input[idx])
        idx += 1
        s = int(input[idx])
        idx += 1
        e = int(input[idx])
        idx += 1

        # 初始化 expenses 数组
        expenses = [[float('inf')] *
(c + 1) for _ in range(n)]
        visited = [[False] * (c + 1)
for _ in range(n)]

        # 优先队列, 元素为 (money, city,
fuel)
        heap = []
        expenses[s][0] = 0
        heapq.heappush(heap, (0, s,
0))

        found = False
        result = -1

        while heap:

```

```

        current_money, city, fuel
    = heapq.heappop(heap)

    if city == e:
        result = current_mone
y
        found = True
        break

    if visited[city][fuel]:
        continue
    visited[city][fuel] = Tru
e

    # 尝试加一升油
    if fuel < c:
        new_fuel = fuel + 1
        new_money = current_m
oney + oil_price[city]
        if new_money < expens
es[city][new_fuel]:
            expenses[city][ne
w_fuel] = new_money
            heapq.heappush(he
ap, (new_money, city, new_fuel))

    # 尝试前往相邻城市
    for (next_city, required_
fuel) in adj[city]:
        if fuel >= required_f
uel:
            new_fuel = fuel -
required_fuel
            if current_money
< expenses[next_city][new_fuel]:
                expenses[next
_city][new_fuel] = current_money
                heapq.heappus
h(heap, (current_money, next_city, ne
w_fuel))

    if found:
        print(result)
    else:
        print("impossible")

if __name__ == "__main__":
    main()

```